

Graphs

Cycle detection, bipartite check, topological sort and shortest paths, explained the way I'd explain it on a whiteboard.

- Cheat sheet
- Basics
- What is a cycle
- Undirected cycles
- Bipartite
- Directed cycles
- Kahn's algo
- Shortest path

Cheat sheet: what each traversal needs

Task	You need
BFS / DFS, undirected	Visited array
BFS / DFS, directed	Visited array
Cycle, undirected, BFS	Visited array + parent stored in the queue as a tuple (node, parent)
Cycle, undirected, DFS	Visited array + parent passed as a method parameter
Cycle, directed, BFS	Kahn's algorithm (indegree). If any node is left over, there is a cycle
Cycle, directed, DFS	Visited array + onPath tracker array

Always loop over every node (`for i in range(n)`), because the graph may be disconnected.

Basics

- Undirected graph:** the edge exists in both directions between vertices (u, v).
- Directed graph:** the edge exists in one direction only, from u to v.
- Path:** a finite sequence of edges (reachable vertices) without repeating a vertex.
- Degree (undirected only):** the number of edges on a vertex. The total degree of the graph equals **2 × the number of edges**.
- Indegree (directed):** number of incoming edges. **Outdegree:** number of outgoing edges.

Tree: a connected graph without a cycle

Tree: a connected graph without a cycle.

What does "seen" mean in traversal? DFS/BFS was already applied to this node. A node can be reached by many paths, so if it is seen, you are reaching it by a different path than before.

Seen vs cycle? Seen only says the node was explored. Finding a cycle needs a separate mechanism on top of it (parent for undirected, onPath for directed).

What is a cycle?

You go from vertex A to vertex B using a path. If there is **another** path that gets you **back** to A from B, the graph has a cycle.

The word **"back"** is the whole point. Before the precise definition, the graph's type matters: a picture of A and B with two arrows can't be answered as "cycle or not" until you know whether it is directed or undirected.

Cycle in Graph
If we move from V_A to V_B using path p_1 , and if there exist a path p_2 (not p_1), to go back to V_A from V_B , the graph has a cycle.

Mathematical Representation of Undirected G.

Undirected Graph: $A \text{ --- } B$

Maths form: $A \leftrightarrow B$

Adj List form: $\begin{cases} A : B \\ B : A \end{cases}$

When we think about Undirected graph, think only in Undirected form $\rightarrow A \text{ --- } B$ (do not confuse)

Does this graph has cycle?

$A \leftrightarrow B$ -? $\rightarrow$ yes $\times$ $\rightarrow$ no

(cannot answer without more info. Directed/Undirected?)

Graph has cycle or not depends on what type of graph is given. Cycle find are diff for Directed and Undirected Graphs

Cycle

Directed

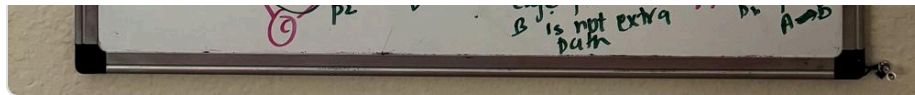
$A \rightarrow B$ $\rightarrow$ yes, cycle exist

Undirected

$A \leftrightarrow B$ $\rightarrow$ no cycle bcoz for undir graph this path is same as one single path. this is just maths representation. we need another path. A $\rightarrow$ B $\rightarrow$ A

Adding multiple edges from A to B

Minimum 3 nodes are required to have cycle in Undirected graph

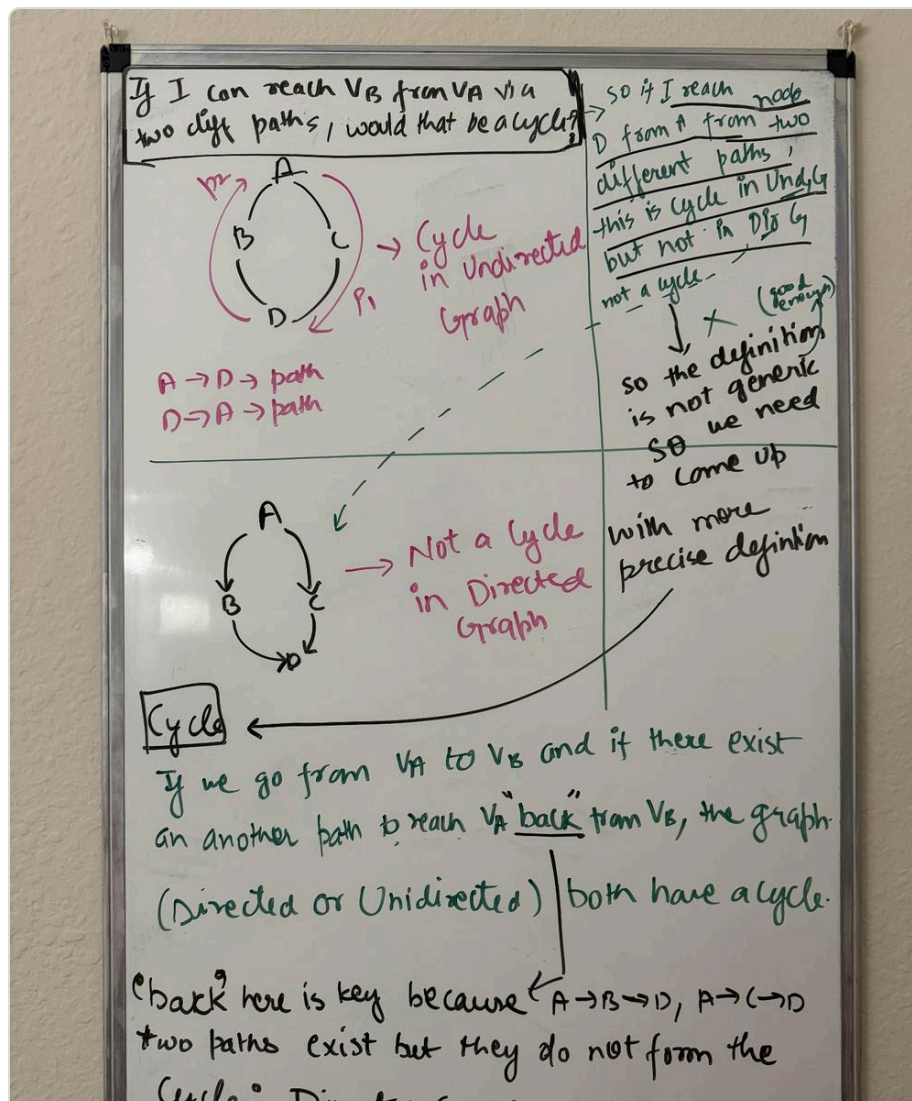


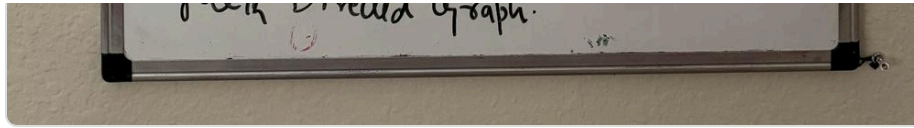
Cycle basics: an undirected edge $A-B$ is stored as $A \rightarrow B$ and $B \rightarrow A$, but that is not a cycle. You need at least 3 nodes.

- **Directed:** $A \rightarrow B$ and $B \rightarrow A$ gives a cycle (2 nodes are enough).
- **Undirected:** $A-B$ is one single path. The two arrows are only the math/adjacency-list representation. Extra parallel edges between A and B are not an extra path. The smallest cycle needs **3 nodes** ($A-B$, $B-C$, $C-A$).
- When you think about an undirected graph, think only in its undirected form. Don't confuse it with the two-arrow form.

Trap: "two different paths to the same node means a cycle"

Not precise. Take $A \rightarrow B \rightarrow D$ and $A \rightarrow C \rightarrow D$. In an **undirected** graph that is a cycle. In a **directed** graph it is not, because you cannot come **back** to A. So the definition with "back" is the generic one.





Two paths to D exist in both graphs, but only the undirected one has a cycle.

Cycle in an undirected graph

The DFS and BFS algorithms for finding cycles are **different** for directed and undirected graphs. For undirected you need only 3 things:

1. **Visited** map. If not visited, mark it and go to that neighbour.
2. **Parent** (the node you came from). Pass it along so you don't walk straight back. If a neighbour is visited **and is not the parent**, that is a cycle. The parent is always visited because it came first, so a visited non-parent means you are reaching a node for the second time by another route.
3. **Loop over all nodes** for disconnected graphs.

DFS

Pass the parent as a parameter of the dfs method. [Video with code](#)

BFS

Put the parent in the queue as a pair (tuple). [Video with code](#)

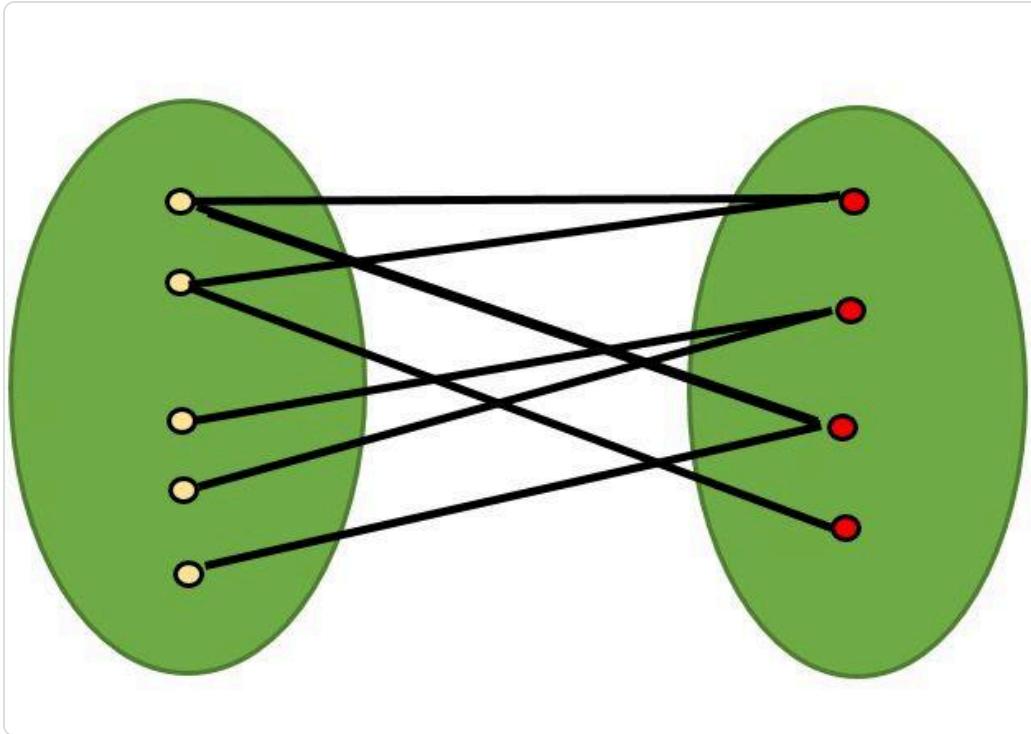
Time complexity

Each node iterates over its edges (its degree), and the sum of degrees is $2E$. Total: $O(V + 2E) = O(V + E)$.

Graph bipartition

A graph is bipartite if its nodes can be split into two independent sets A and B so that **every edge connects a node in A to a node in B**.

- Graphs with no cycles (linear graphs, trees) are always bipartite.
- A graph whose cycles are all of even length is bipartite.
- A graph with an odd-length cycle can **never** be bipartite.



Every edge goes across: yellow set to red set, never inside a set.

Why only for undirected graphs? Colouring needs a path to every node. A directed edge gives you only one way across. Example: edges $[A, B], [C, B]$ is $A \rightarrow B \leftarrow C$. Colour A red, then B yellow, but you can never reach C from B, so C stays uncoloured (-1). So convert the input to an undirected graph first. [Explanation on LeetCode](#)

BFS colouring (Is Graph Bipartite?)

`seen` does double duty: it is the colour **and** the visited flag (-1 = unvisited, $0/1$ = colours). Because a coloured node is already seen, you never add it to the queue again.

```
from collections import deque

class Solution:
    def check(self, graph, i, seen):
        q = deque([i])
        seen[i] = 0
        while q:
            node = q.popleft()
            for nei in graph[node]:
                if seen[nei] == seen[node]:    # same colour on both ends
                    return False
                if seen[nei] == -1:            # uncoloured: give opposite colour
                    seen[nei] = 1 - seen[node]
                    q.append(nei)
        return True
```

```
def isBipartite(self, graph):
    seen = [-1] * len(graph)
    for i in range(len(graph)):
        # disconnected graphs
        if seen[i] == -1 and not self.check(graph, i, seen):
            return False
    return True
```

Cycle in a directed graph

Why the undirected algorithm fails

The undirected logic: walk a path, explore neighbours, use a parent check to avoid stepping back, and if you meet a node that is already visited and not the parent, a cycle exists. In a directed graph this says "cycle" even when there isn't one.

Why we cannot use the same BFS/DFS Undirected graph algorithms for Directed Graph for Cycle Detection?

ADG "with Cycle" vs DG "without Cycle"

Logic in ADirectedGraph.
 We go on a path, explore neighbours, have a parent check, to avoid going back to same path (bypassing prev node), and if we see a node (already visited and not the parent) → means → cycle → some path has already discovered that node.

But, in case of Directed Graph, this algo will say 'cycle' even if it's not.

When applying DFS from '1' it will explore 1 → 2 → 3 → 4 → 5 → 6 → 1. When it reaches 6 and try to explore its neighbours, it will say '1' is not my parent (parent is 5), and '1' is seen before.

Graph has a cycle → Wrong!!

DFS from 1 goes $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, comes back and goes $5 \rightarrow 6 \rightarrow 4$. Node 4 is seen and is not 6's parent, so it says "cycle". Wrong!

DFS: visited + onPath

- **seen** is the same as visited.
- **onPath** tracks the nodes of the *current* path. Add a node as DFS goes in, remove it on the way back (like backtracking).
- **Detect a cycle:** if you see an **onPath** node again, the graph has a cycle.
- **Why visited too?** The path grows and shrinks as DFS runs. In a disconnected graph you don't want to run DFS again from nodes that are already visited.
- **Tracking the path:** instead of passing a path list into every call, keep one array and set/unset it as DFS expands or retreats.

Cycle Detection in Directed Graph - (DFS)

Need 2 things

- 1) **seen** (same as Visited)
- 2) **onPath** (list)

onPath: - We need to keep track of nodes of current path and while coming back removing the node from the current path (like in backtracking)

How to detect a cycle?
If you see a 'onPath' node again (twice), graph has cycle.

Why we need visited also?
Path will expand and shrink as DFS goes. Let's say if graph is disconnected, we do not want to apply DFS on visited nodes again.

How to keep track of Path?
Rather than passing path on each dfs, we can create an path array and set/unset as DFS expands or retreats.

Algo:

```

def dfs(node, G):
    self.seen[node] = True
    self.onPath[node] = True
    for nei in G[node]:
        if self.onPath[nei] == True:
            return False (cycle)
        elif not self.seen[nei]:
            dfs(nei, G)
    self.onPath[node] = False
    return True
  
```

reset

(DFS Directed Graph Template)

Logic 2 explore if onPath[nei] == True: return False (cycle)
else if not visited: (DFS)

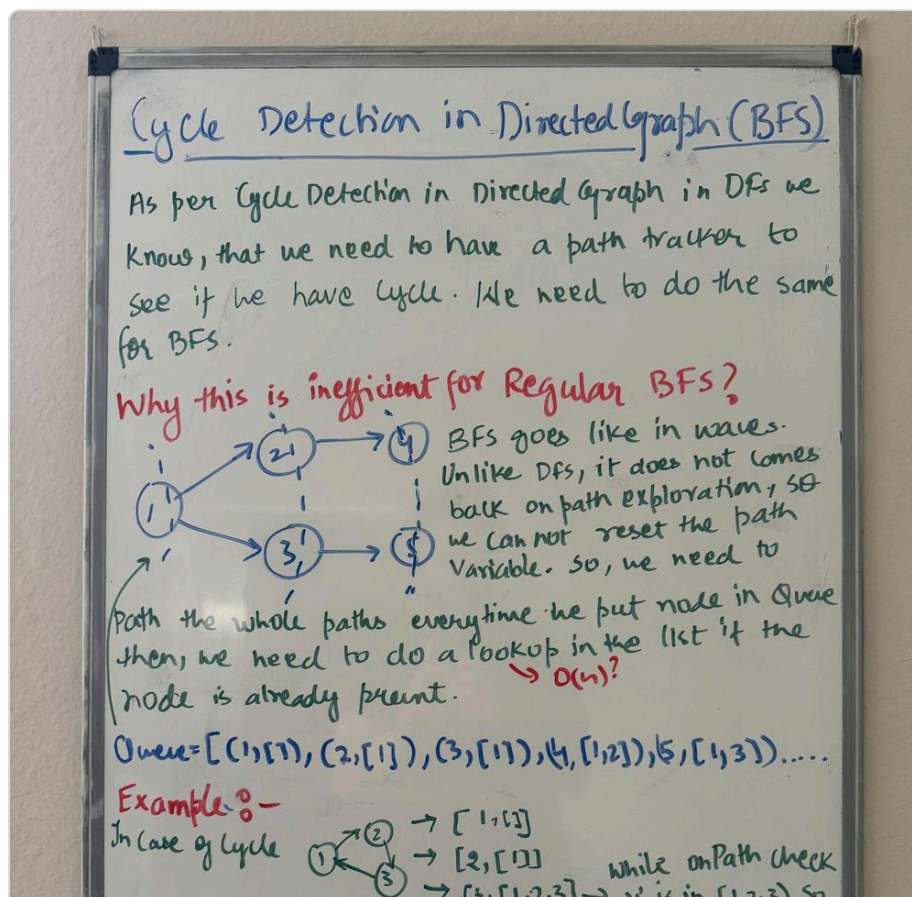
DFS directed-graph template. Path [1,2,3,4,1]: node 1 comes again while still on the path.

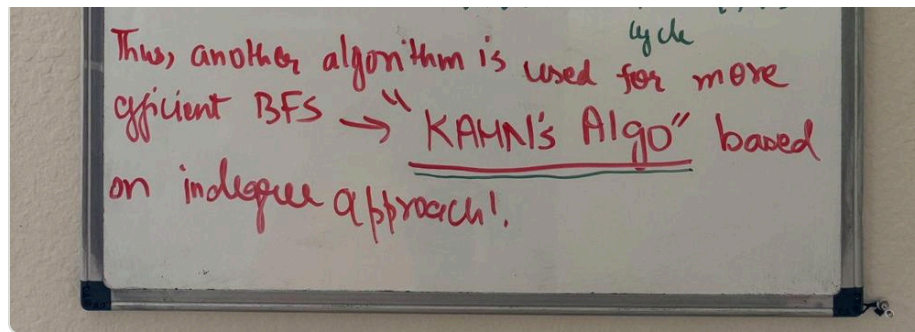
```
def dfs(node, G):
    seen[node] = True
    onPath[node] = True                # set
    for nei in G[node]:
        if onPath[nei]:                # already on current path: cycle
            return False
        if not seen[nei]:
            if not dfs(nei, G):         # pass the "cycle found" result upward
                return False
    onPath[node] = False               # reset while backtracking
    return True                        # no cycle from this node
```

Time complexity: $O(V + E)$. Each node is visited once and each directed edge is explored once.

BFS: why the path tracker is a bad idea

The same idea (a path tracker) would work for BFS, but it is inefficient. BFS moves in waves and never comes back along a path, so you can't reset the path variable. You would have to store the whole path with every node you push, and then search that list for the node, which costs $O(n)$ each time.





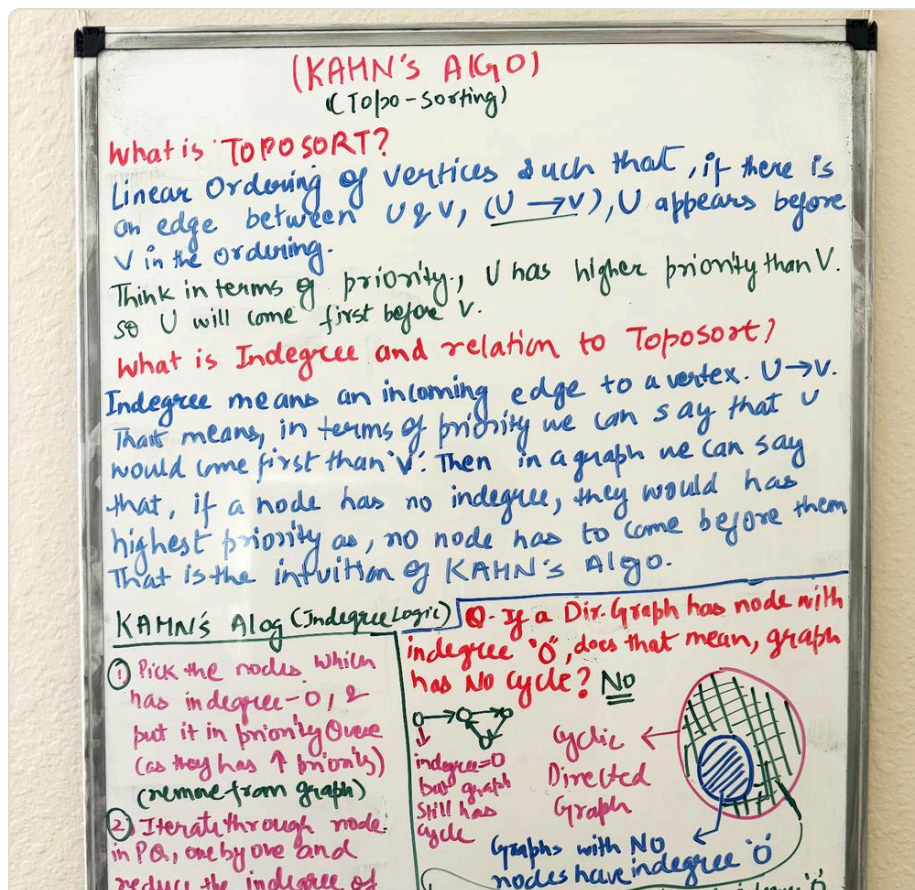
Queue of (node, path) pairs. For $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ the path $[1, 2, 3]$ already contains 1, so it is a cycle, but the cost is too high.

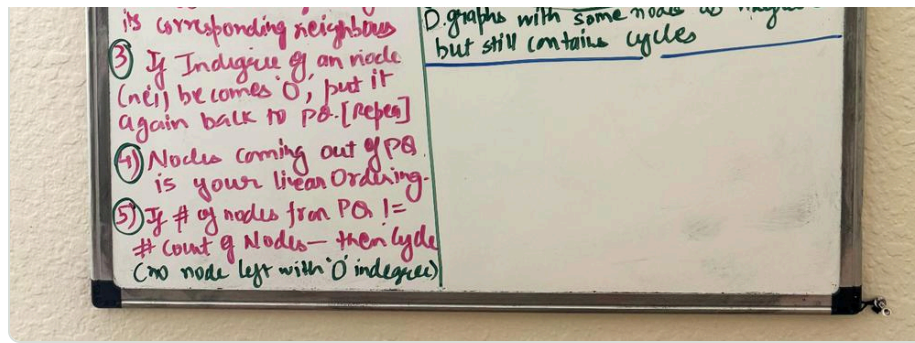
So a more efficient BFS approach is used: **Kahn's algorithm**, based on indegree.

Topological sort and Kahn's algorithm

Topological sort: a linear ordering of vertices such that for every edge $U \rightarrow V$, U appears before V . Think of it as priority: U has higher priority than V , so U comes first.

Indegree and topo sort: an incoming edge $U \rightarrow V$ means U must come before V . A node with indegree 0 has nothing that must come before it, so it has the highest priority. That is the intuition of Kahn's algorithm.





Kahn's algorithm: steps, and why indegree 0 does not mean "no cycle".

1. Pick all nodes with indegree 0 and put them in the queue (they have top priority). Remove them from the graph.
2. Take nodes from the queue one by one and reduce the indegree of their neighbours.
3. If a neighbour's indegree becomes 0, add it to the queue. Repeat.
4. The order in which nodes come out of the queue is your linear ordering.
5. If the number of nodes that came out is **not equal** to the total number of nodes, there is a cycle (some nodes never reached indegree 0).

Q: If a directed graph has a node with indegree 0, does that mean it has no cycle? No. A graph can have some nodes with indegree 0 and still contain a cycle elsewhere (e.g. a node pointing into a triangle).

Theorem: if every node of a directed graph has a positive indegree, the graph contains a cycle. The reverse is **not** true, as the box above shows.

DFS topo sort: run DFS, push each node onto a stack after its dfs call completes, then empty the stack to get the order.

Shortest path algorithms

- **Undirected, unit weights:** BFS. Same algorithm as shortest path in a matrix.
- **DAG (directed acyclic graph):** topo sort with DFS, then relax edges in that order. It only works for a DAG because topo order exists only without cycles.
- **Dijkstra (non-negative weights):** always finalise the vertex with the shortest known distance, then update the distances of its neighbours through it.

After Dijkstra finishes, the parent links form a **shortest-path tree**: a spanning tree where the path from the source to every node is the shortest one. This is not the same as a minimum spanning tree (that is Prim's).

[Proof of correctness for Dijkstra \(video\)](#).

Still to add

Bellman-Ford, Prim's, Union-Find, and DFS vs BFS complexity on multi-edge graphs.

Graphs revision notes. Whiteboard photos are my own.